Sustainable Programming – Softwarecode ohne Stromfresser

Quelle: https://www.heise.de/developer/artikel/Sustainable-Programming-Softwarecode-ohne-Stromfresser-4197828.html?seite=all

30.10.2018

Wie man Anwendungssysteme dazu bringt, den Energieverbrauch einer ganzen Stadt zu sparen.

Green IT ist ein alter Hut – könnte man meinen. Wikipedia zum Beispiel datiert die Anfänge auf 1992. Damals hatte die US-Umweltbehörde EPA gerade den Energy Star herausgebracht, ein Gütesiegel, um stromsparende Endgeräte zu kennzeichnen. Heute, gut 25 Jahre später, beherrscht die grüne Cloud die Diskussion. Dabei lautet die Kernfrage, wie sich Rechen- und Speicherleistungen umweltschonend bereitstellen lassen.

Nach wie vor konzentriert sich Green IT auf den Betrieb und die Nutzung von Applikationen. Welche Weichen in ihrer Entwicklung gestellt werden, gerät eher weniger in den Fokus. Software-Ingenieure bewegen sich da oftmals wie in einer Art Elfenbeinturm. Dabei lässt sich mit geringem Aufwand eine Menge erreichen. Häufig sind es recht überschaubare Weichenstellungen, die dann im Wirkbetrieb zu erheblichen Verbrauchsunterschieden führen, die etwa bei größeren B2B-Anwendungen dem Jahresverbrauch mittelgroßer Städte entsprechen können.

Was beim CPU-Verbrauch drin ist

Referenz ist eine typische Rechenzentrumsumgebung für transaktionsstarke B2B-Applikationen. Wird in einer solchen Umgebung mit einer erfolgreich implementierten Sustainable-Programming-Maßnahme eine CPU-Sekunde eingespart, entspricht das einer Energiereduktion von 10 Wattsekunden pro Transaktion. Wird eine solche Transaktion von 1,5 Millionen Anwendern an 230 Werktagen 20-mal täglich durchgeführt, summiert sich die Energieersparnis auf 19 Megawattstunden. Wenn nun tausend Entwickler fünf Transaktionen

auf diese Weise optimieren, kommt man gemäß Stromspiegel der Bundesregierung auf den Jahresverbrauch von über 30.000 Zwei-Personen-Haushalten.

Gleichwohl ist Sustainable Programming noch eine junge Disziplin, so man denn überhaupt von einer Disziplin sprechen kann. Denn wie sehr man damit Neuland betritt, zeigt eine simple Internetabfrage. Ganz gleich welche Suchmaschine man hierzu einsetzt, es ergibt sich kaum ein Treffer, der darauf hindeuten würde, dass sich energieeffiziente Softwareentwicklung als Best Practice etabliert hätte.

Um trotzdem kurzfristig etwas zu erreichen, ist zunächst einmal Pragmatismus gefragt. Bei der SAP wurde daher vor etwa zehn Jahren damit begonnen, spezifische Handlungsempfehlungen für das Engineering zu entwickeln, die zu mehr Energieeffizienz führen. Das Unternehmen baut damit auf einem Programm zur effizienten Nutzung von Hardware auf, welches das interne Qualitätsmanagement seit den Neunzigerjahren betreibt. Dessen Kernziel war und ist es, die Leistungsfähigkeit der Systeme bei den Antwortzeiten und beim Datendurchsatz zu optimieren.

Implizit führen die Maßnahmen des Programms fast immer auch zu einer höheren Energieeffizienz. Vor dem Hintergrund stellte der Stromverbrauch zunächst keine explizite Zielgröße dar. Vor etwa zehn Jahren hat sich das geändert. Seither wurde der Blickwinkel des Programms erweitert und den Faktoren Performance und Skalierbarkeit die Energieeffizienz als gleichrangige Kernanforderung an die Seite gestellt. Man hatte nämlich festgestellt, dass es spezifische Anwendungsszenarien gibt, in denen sich die drei Faktoren nur dann wunschgemäß entwickeln, wenn man ganz bewusst auch den späteren Stromverbrauch mit auf den Radar nimmt.

Aus diesem Wissen hat die SAP Handlungsempfehlungen für die Entwicklung abgeleitet. Diese ersten pragmatischen Schritte erheben keinerlei Anspruch darauf, einen umfassenden Ansatz dafür zu liefern, wie sich energieeffiziente Softwareentwicklung in jedweder Organisation verwirklichen lässt. Vielmehr dienen sie als Beispiele, um im Folgenden die Denk- und Handlungsprinzipien zu erläutern, die dem Sustainable Programming zugrunde liegen.

Unorthodoxe Mittel

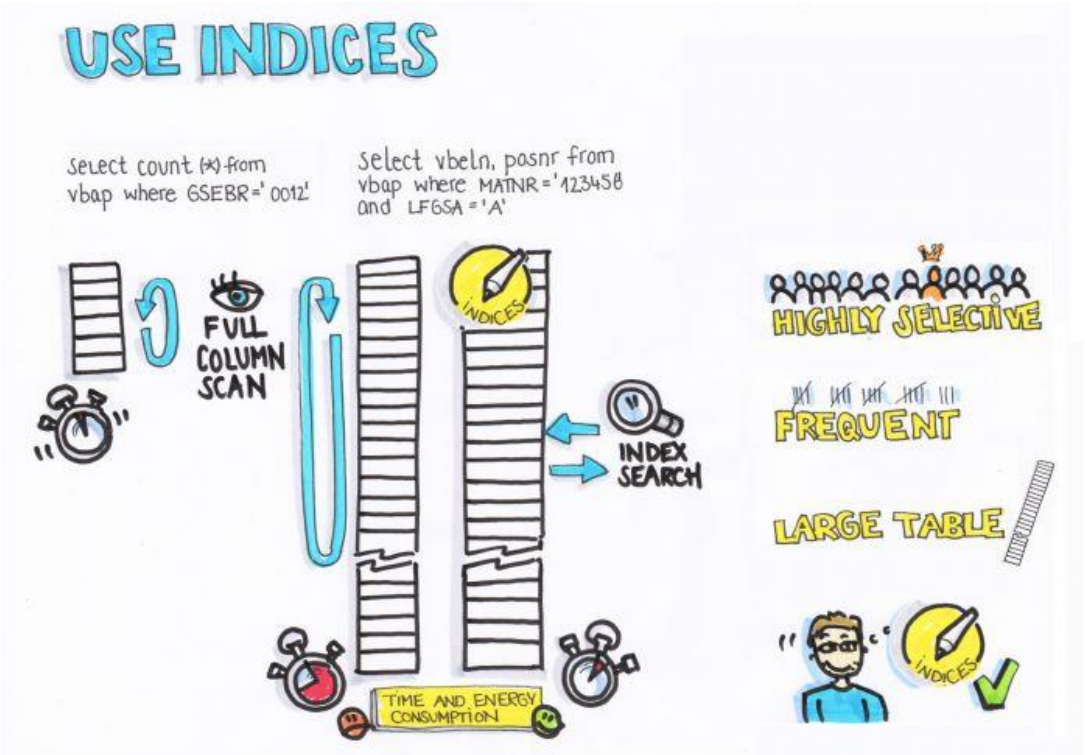
Denn die eigentliche Hürde der energieeffizienten Softwareentwicklung ist keineswegs technischer, sondern vielmehr mentaler Natur. Sie besteht darin, von Fall zu Fall den Entschluss zu fassen, die gewohnten Engineering-Wege zu verlassen und zu alternativen Mitteln zu greifen, wenn sich damit in nennenswerter Weise Energie sparen lässt. Gemeint sind damit insbesondere auch solche Mittel, die auf den ersten Blick eher unorthodox erscheinen. Im Fall der SAP ist In-Memory Computing ein gutes Beispiel dafür. Denn wenn die Datenverarbeitung nur noch im Hauptspeicher läuft, erscheint ihr Performanceversprechen geradezu unbegrenzt. Sich hier noch mit Energie- oder gar Laufzeitoptimierung zu beschäftigen, kommt manchem dann beinahe müßig vor.

Ungeachtet dessen treten jedoch in der In-Memory-Welt Anwendungsfälle auf, die ein Abweichen vom üblichen Vorgehen rechtfertigen. Zum Beispiel durch das Setzen von Indizes. Wie aber kann das sein, werden einen Kollegen vielleicht fragen. Indizes sind schließlich nur ein Hilfsmittel, um in herkömmlichen, also zeilenbasierten Datenbanken Performancedefizite auszugleichen. Schließlich machen solche Datenbanken zunächst den Scan der gesamten Tabelle erforderlich, was zu deutlich höherem Rechenaufwand führt. Um die Antwortzeiten zu optimieren, setzt man einen Index auf die Spalte, die für die anstehende Abfrage gebraucht

wird. Der Index dupliziert die Spalte und sortiert ihre Inhalte im Sinne der Abfrage neu. In der Folge verkürzen sich die Antwortzeiten.

In der In-Memory-Welt erscheint ein solcher Workaround antiquiert. Schließlich führt die spaltenbasierte Struktur solcher Datenbanken dazu, dass jede Spalte ohnehin ein Index ist. Warum also sollten sich Entwickler noch weiter Gedanken über den Einsatz von Indizes machen? Tatsächlich gibt es aber Anwendungsfälle, in denen ihr Einsatz einen echten Zusatznutzen bringt. Denn spannend wird es immer dann, wenn man es mit extrem großen Spalten zu tun hat, auf die man in hoher Frequenz selektiv zugreift. Etwa in OLTP-Systemen, die überwiegend hohe Volumen mit gleichen und sich wiederholenden Abläufen verarbeiten. Wer in einem solchen System einen einzelnen Auftrag unter den oben genannten Prämissen abfragt, dem ergeben sich Antwortzeiten, die selbst in einer In-Memory-Architektur spürbar werden, sodass die Betreiber recht bald über den Einsatz zusätzlicher Hardware nachdenken, um ihre SLAs zu halten.

Deutlich nachhaltiger, und zwar sowohl aus ökologischer als auch aus ökonomischer Sicht, wäre es, einen Index zu implementieren, sodass kein kompletter Spalten-Scan erforderlich ist. Die damit einhergehende Ressourceneinsparung wurde für eine Tabelle mit 100 Millionen Einträgen kalkuliert. Ohne Index hätte der Energieverbrauch bei 5,4 Wattsekunden pro Abfrage gelegen. Mit dem Index kam er auf 0,035 Wattsekunden. Das entspricht einer Reduktion um den Faktor 155.



Auch in der In-Memory-Welt kann es Anwendungsfälle geben, bei denen der Gebrauch von Indizes zu erheblichen Ressourceneinsparungen führt. (Bild: SAP SE)

Rechenanforderungen steigen – Energieverfügbarkeit sinkt

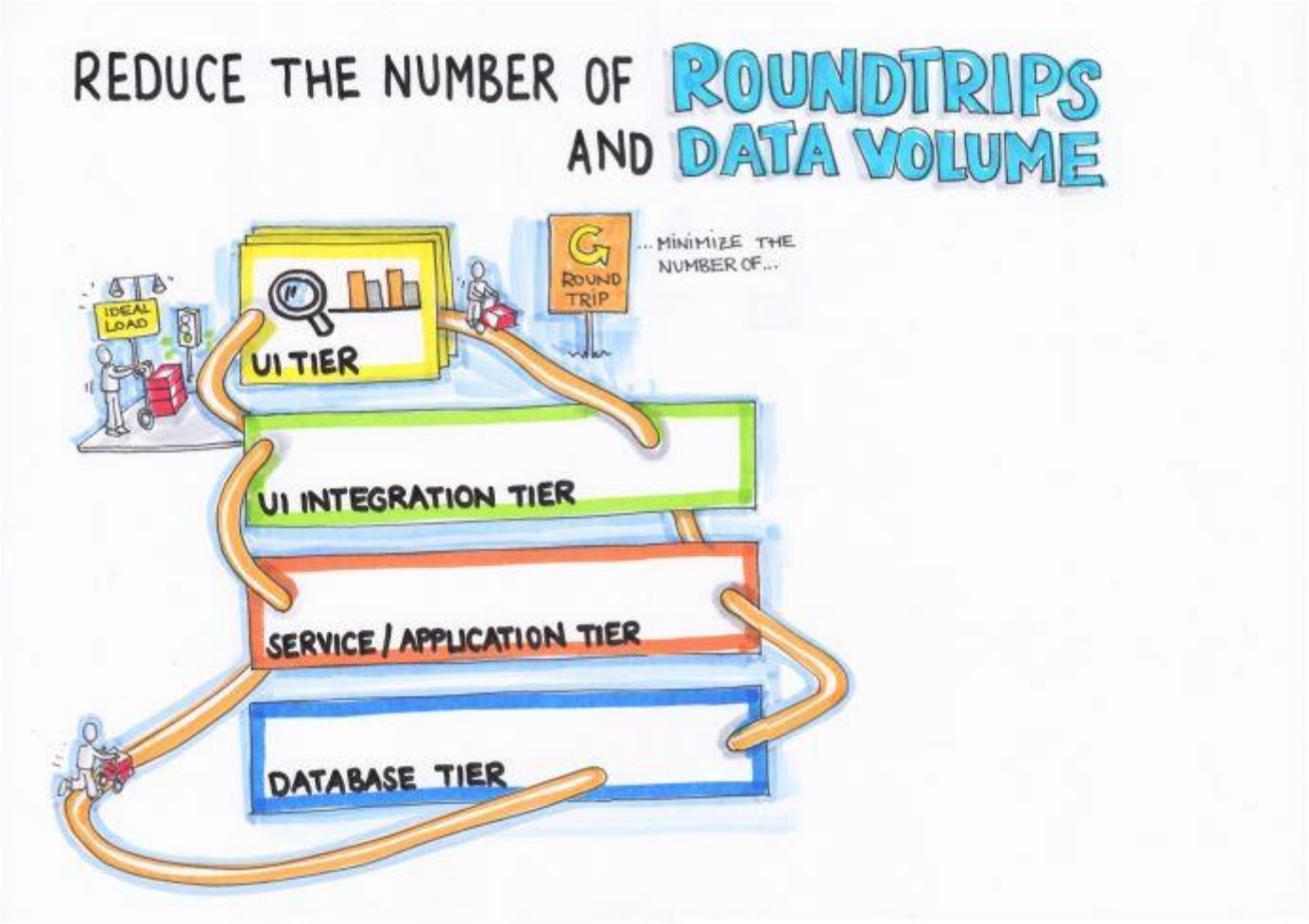
An der Stelle mag der Einwand kommen, dass solche Abfragen doch wohl eher die Ausnahme sind. Wie sinnvoll ist es daher, sich mit der derlei Eventualitäten zu beschäftigen? Angesichts

der Veränderungen, die die Digitalisierung hervorruft, werden sich die Zweifel recht bald erübrigen. Mit beachtlichem Tempo bewegen sich Unternehmen in eine Anwendungswelt hinein, in der die Zahl hochspezifischer Abfragen geradezu sprunghaft zunimmt. Denkt man die aktuellen IoT-/Industrie-4.0-Szenarien konsequent zu Ende, werden die Losgrößen, mit denen man es in Zukunft zu tun hat, massiv abnehmen. Etwas zugespitzt lässt sich sagen, dass die Systeme dann nicht mehr nur den einen Kundenauftrag mit vielleicht tausend Positionen, sondern tausend Aufträge mit jeweils nur einer einzigen Position bearbeiten müssen.

Die Architekturen müssen sich somit völlig neuen Herausforderungen stellen. Übrigens auch mit Blick auf das Durchhaltevermögen der beteiligten Endgeräte. Immer weniger davon werden einen dauerhaften Zugang zum Stromnetz haben. In dezentralisierten IoT-Anwendungen wird es daher umso stärker darauf ankommen, dass sich die Rechenarbeit mit einem minimalen Stromverbrauch erledigen lässt.

Intelligente Nutzung von Caches

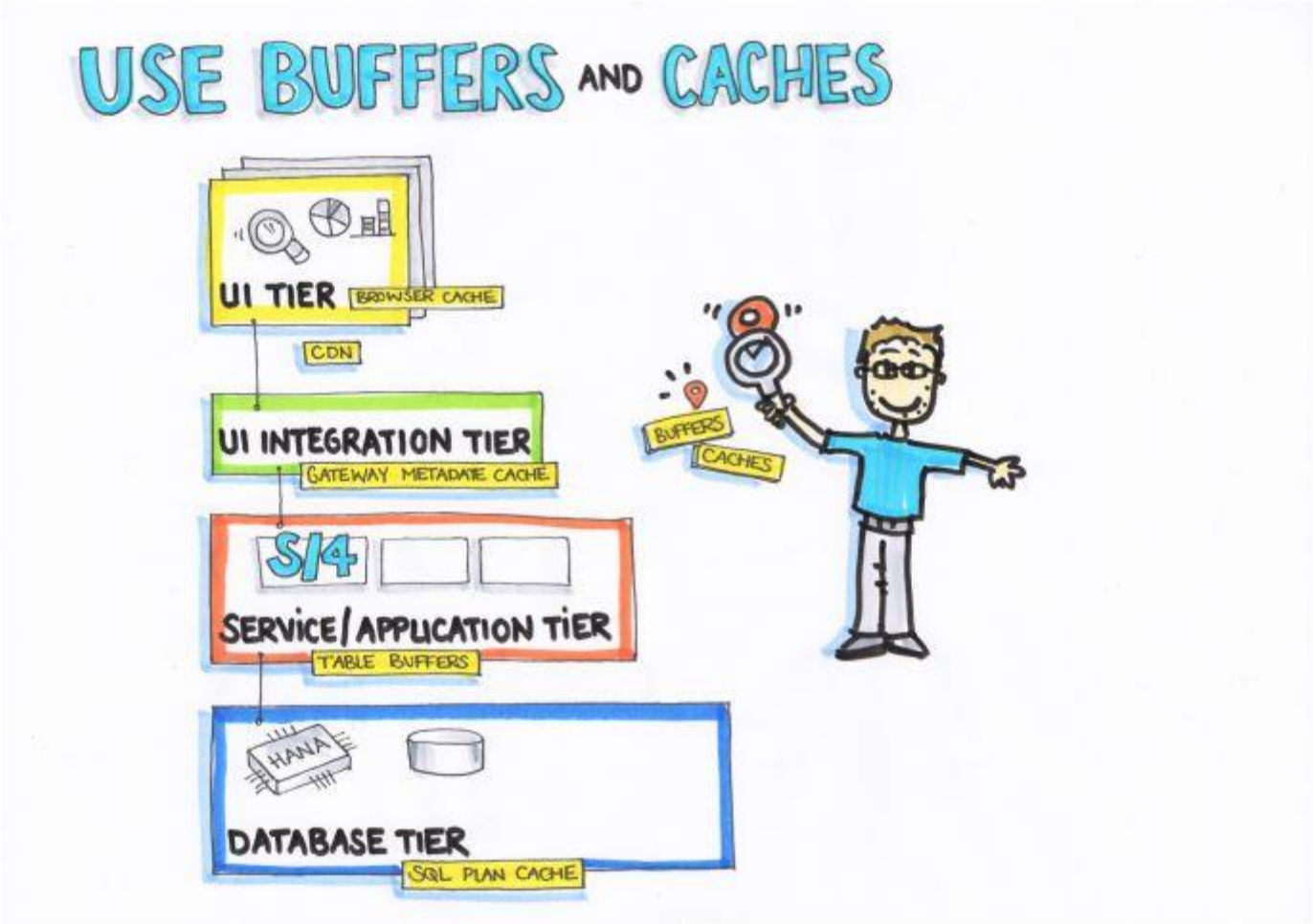
Softwareentwickler sollten sich daher deutlich stärker als bisher mit dem Thema Caches beschäftigen, und zwar entlang der gesamten Systemarchitektur, auf der die Anwendungen laufen. Das Ziel liegt stets darin, die erforderlichen Daten möglichst schon dort zu puffern, wo die Rechenaufgaben entstehen. Nur dann lassen sich energie- und zeitintensive Roundtrips durch den kompletten Stack einer Systemlandschaft vermeiden.



Caches bieten eine Vielzahl von Möglichkeiten, energieintensive Roundtrips zu vermeiden. (Bild: SAP SE)

Das beginnt bereits im Frontend mit dem Implementieren geeigneter Browser-Caches. Im Netzwerk wiederum sind Content Delivery Networks das Mittel der Wahl. Sie erlauben es, Daten und Dokumente so nah wie möglich bei den darauf zugreifenden Komponenten vorzuhalten. Für den Fall, dass es sinnvoller ist, den Request erst im Backend zu verarbeiten, bietet sich die Nutzung der Cache-Infrastruktur des Applikationsservers an, um Metadaten oder häufig verwendete Anwendungsdaten zu "cachen".

Auf der Datenbankebene wiederum empfiehlt sich der Einsatz von SQL Plan Caches an. Hierin werden Ablaufpläne gespeichert, die frühere SQL-Anweisungsausführungen erzeugt haben. Wird ein solcher Ablaufplan, das heißt der optimale Zugriffspfad, wiederverwendet, lässt sich damit eine Menge Energie sparen. Ein typischer Energieverbrauch für die Anweisungsvorbereitung beträgt zwei Wattsekunden – für sich betrachtet sicherlich ein eher marginaler Wert. Da aber zahlreiche Produktivsysteme mit 100.000 Statements pro Sekunde arbeiten, ergeben sich in der Summe recht attraktive Einsparpotenziale.



Caches eignen sich sowohl auf Frontend- und Netzwerk- als auch auf Applikations- und Datenbankebene. (Bild: SAP SE)

Dynamic Result Caches

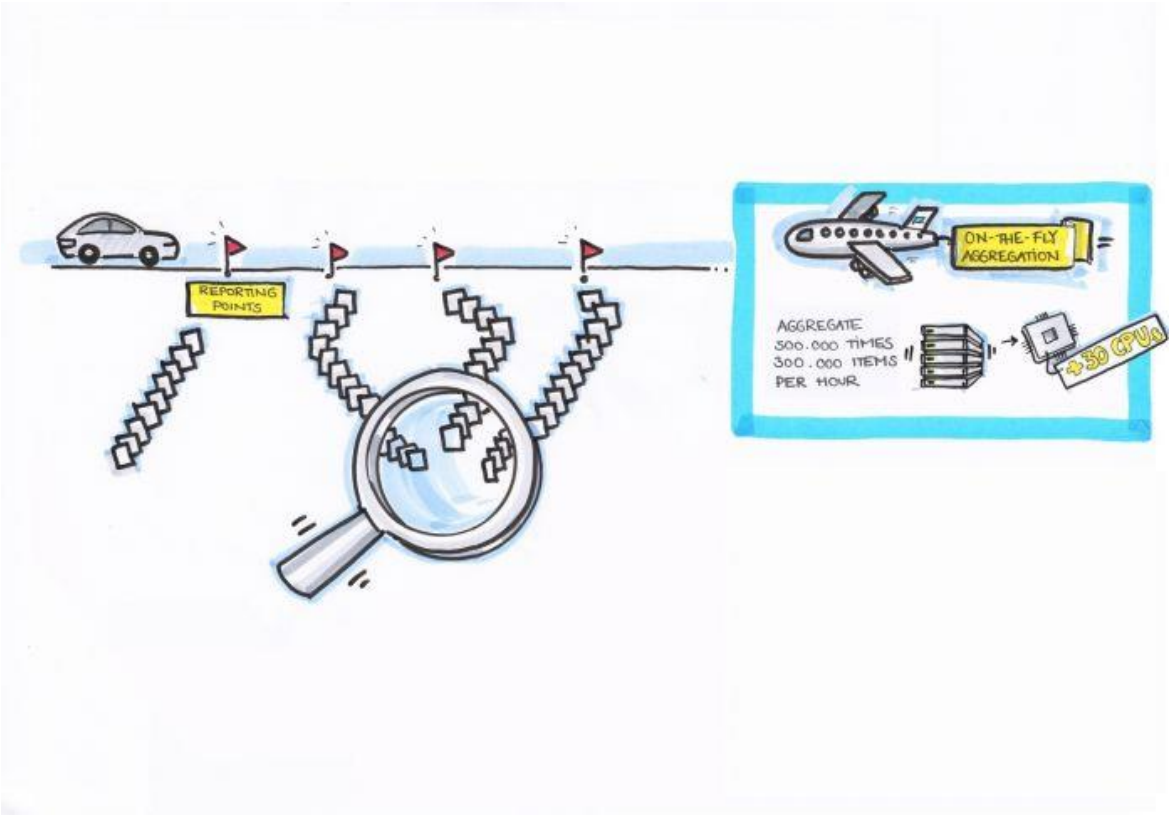
Nun könnte man sagen, dass der Einsatz von Caches doch ohnehin zum Alltag intelligenter Softwareentwicklung gehört. Schließlich ist es sowieso das Ziel, performante Anwendungen zu entwickeln. Dass parallel dazu noch besonders energieeffiziente Systeme entstehen, ist sicherlich willkommen, muss aber eigentlich nicht mehr extra erwähnt werden. Doch ähnlich wie bei der oben dargestellten Nutzung von Indizes erweist sich hier die Praxis deutlich vielschichtiger, als zunächst angenommen.

Recht eindrucksvoll zeigt das ein weiteres Beispiel aus dem In-Memory Computing. Um es aus der Perspektive der Entwickler zu verstehen, sei zunächst in die Zeit vor SAP HANA geschaut. In reinen R3-Umgebungen galt es als probates Mittel, bestimmte Datenbankinhalte noch einmal zusätzlich in sogenannten Aggregattabellen aufzubereiten. Das betraf jene Inhalte, die gebraucht wurden, um in ausgewählten Anwendungsfällen in möglichst kurzer Zeit zum Beispiel einen Gesamtbestand oder -bedarf zu ermitteln. Etwa wenn der Einkauf den Wert aller offenen Bestellungen ermitteln wollte, ohne hierzu sämtliche Kundenaufträge zu selektieren. Das Vorgehen hatte natürlich seinen Preis. Jedes Mal, wenn ein neuer Kundenbeleg hereinkam, war neben der Tabelle mit dem Kundenbeleg auch die Aggregattabelle zu aktualisieren. In-Memory Computing bietet nun die Möglichkeit, ein solches Aggregat nur noch bei Bedarf on the fly zu bilden und dann ad hoc auszuwerten.

Tatsächlich haben Aggregattabellen in bestimmten Fällen aber auch weiterhin ihre Daseinsberechtigung. Denn es gibt zum Teil Anwendungsszenarien, in denen die On-the-fly-

Aggregation zu einem CPU Verbrauch führt, bei dem ein alternatives Vorgehen ratsam ist. Zum Beispiel in der Automobilindustrie, wenn ein Fahrzeug eine Fertigungsstraße durchläuft und parallel dazu zu dokumentieren ist, welche Materialien zu welchem Zeitpunkt verbaut werden. Um die entsprechenden Materialbewegungen vom Lager in die Fertigung zu erfassen, fallen in einem typischen Automotive-Szenario rund eine halbe Million Bestandsabfragen (Inventory Checks) pro Stunde an. Wer an der Stelle nun ausschließlich auf Ad-hoc-Reporting setzt, hat es zwangsläufig mit 500.000 On-the-fly-Aggregationen zu tun – pro Stunde. Da jede Aggregation auf einer Datenbank mit gut und gerne 300.000 Einträgen arbeitet, heißt das, dass 500.000-mal 300.000 Artikel zu aggregieren sind. Stündliche Inventarprüfungen dieser Größenordnung beanspruchen dann bis zu 30 CPU-Kerne.

Vor dem Hintergrund erweist es sich als deutlich nachhaltiger, sogenannte Result Caches einzusetzen, welche die zu verarbeitenden Datenbankinhalte als Aggregat vorhalten und somit an die Stelle der ressourcenaufwendigen On-the-fly-Aggregation treten. Um zusätzliche Performance- und Energieeffizienzgewinne zu erzielen, können die Entwickler entscheiden, wie häufig diese Caches aktualisiert werden sollen, um die Anforderungen der Anwendung zu erfüllen. Je nach Szenario reicht unter Umständen bereits ein On-Demand-Update (Static Result Cache) aus. In besonders zeitkritischen Szenarien wie dem beschriebenen mag es aber auch so sein, dass man das Update sofort braucht (Dynamic Result Cache), wenn ein neuer Beleg eintrifft.

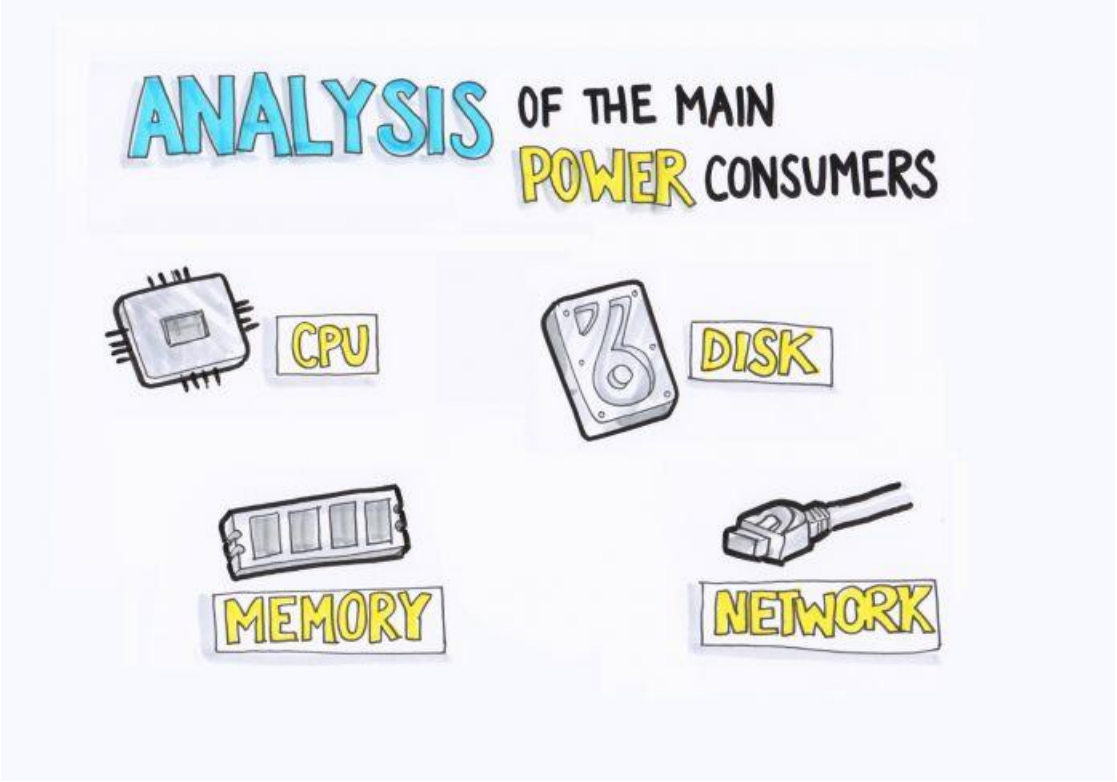


Result Caches halten die zu verarbeitenden Datenbankinhalte als Aggregat vor. Damit eignen sie sich in bestimmten Fällen für Abfragen in extrem komplexen In-Memory-Umgebungen. (Bild: SAP SE)

Ohne Strategie droht ein Verzetteln

Je weiter man sich in die Optionen des Sustainable Programming hineinarbeitet, desto breiter wird das Spektrum an unterschiedlichen Möglichkeiten. Die Vielfalt birgt jedoch immer auch die Gefahr, sich in der Praxis zu verzetteln. Um das zu verhindern, sollte man gleich zu Beginn der Sustainability-Reise eine durchgängige Strategie dafür überlegen, wo Entwickler ansetzen sollen und welche Mittel sich dort anbieten.

Als Ausgangspunkt bietet sich die Überlegung an, dass es in der Softwareentwicklung vier große Energieverbraucher gibt: CPU, Arbeitsspeicher, Festplatte und Netzwerk. Wichtig dabei: Die Energiebedürfnisse dieser vier bedingen einander. Wenn man an einer Stelle den Verbrauch senkt, wirkt sich das immer auch auf den Energiehunger der übrigen drei aus. Sich über den Wirkungszusammenhang Klarheit zu verschaffen, entscheidet über den Erfolg der Sustainability-Maßnahmen.

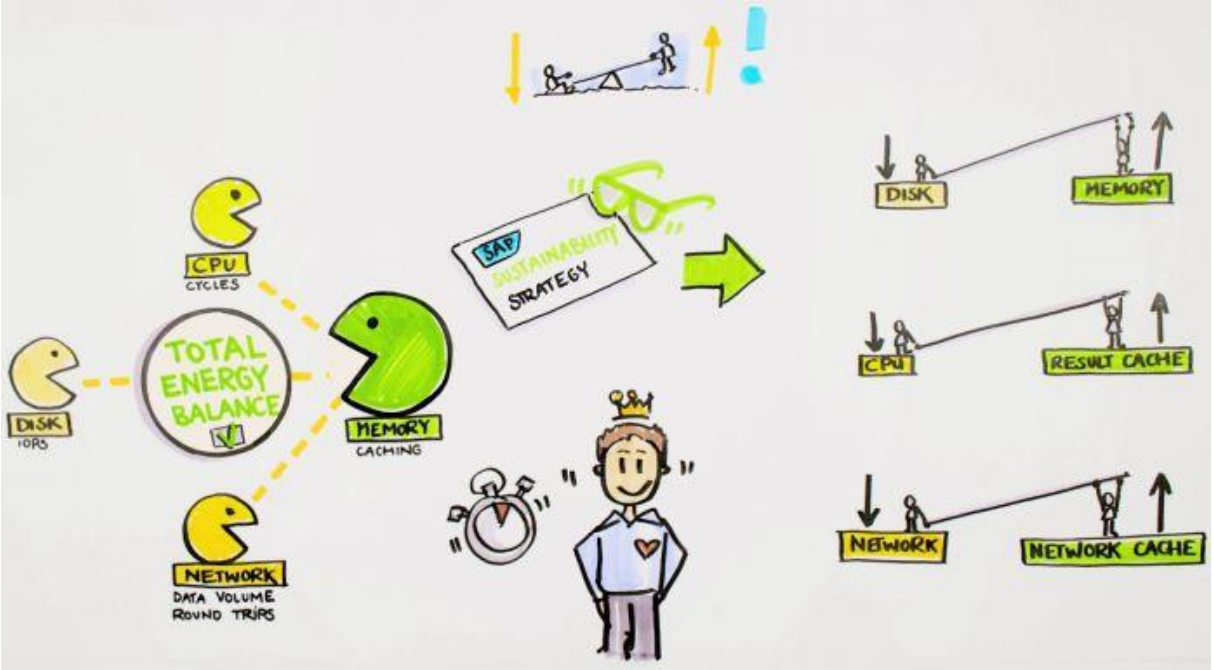


Entwickler haben es mit vier großen Energieverbrauchern zu tun: CPU, Arbeitsspeicher, Festplatte und Netzwerk. (Bild: SAP SE)

Softwareentwickler brauchen daher das Feedback der Betreiber, wie sich ihre Produkte in der Praxis schlagen – zumindest näherungsweise. Somit gewinnen sie eine belastbare Entscheidungsgrundlage für die Strategieentwicklung und wissen, an welchen Stellschrauben sie im Entwickleralltag drehen sollten. Ein einziges übergreifendes Patentrezept für alle Entwicklungsbereiche und -organisationen kann es somit nicht geben. Die Optimierungsstrategie der SAP beruht vor allem auf diesen drei Säulen:

- In-Memory Computing statt Disk IO
- Caches statt CPU-Zyklen

- Content Delivery Network und Code Pushdown statt hohem Datentransfer und hoher Anzahl von Roundtrips



Strategie zur Optimierung der Gesamtenergiebilanz der SAP. (Bild: SAP SE)

Obwohl die Strategie zu einem erhöhten Memory-Energieverbrauch – und damit einhergehend auch zu Mehrkosten für den Arbeitsspeicher – führt, fällt die Gesamtenergiebilanz durch die Einsparungen bei den anderen Komponenten (d. h. CPU, Festplatte und Netzwerk) positiv aus. Gleichzeitig werden die End-to-End-Antwortzeit und die User Experience verbessert.

Keine harten Vorgaben machen

Moderne Systeme bieten zahlreiche Möglichkeiten, um energieeffiziente Software zu schreiben. Es ist jedoch eine Strategie zu erarbeiten, um die vier großen Energieverbraucher in ihrem Zusammenspiel zu optimieren, ohne Performancenachteile zu haben. Ob und wie eine solche Strategie dann mit Leben gefüllt wird, entscheidet sich im Tagesgeschäft. Dass Software-Ingenieure nicht nur mit Features, sondern auch in Sachen Energieeffizienz punkten können, ist ein Lernprozess, der seine Zeit braucht. Im Fall von SAP hat man sich dazu entschlossen, spezifische Trainings zu entwickeln und die Kollegen in der Entwicklung kontinuierlich zu begleiten.

Außerordentlich hilfreich ist es außerdem, wenn die Entwickler erkennen können, dass sie nicht die einzigen im Unternehmen sind, die auf einmal "grün" sein sollen. Dauerhaft werden sie nur bei der Stange bleiben, wenn die von ihnen gewünschten Verhaltensänderungen im Kontext eines Umdenkens stehen, das auf allen Ebenen greift. Gerade deshalb ist es wichtig, dass auch das Management mitzieht und glaubhaft für die Nachhaltigkeitsziele des Unternehmens einsteht. Mit einer solchen Rückendeckung wächst die Bereitschaft, die gewohnten Wege zu verlassen, um im Sinne der Energieeffizienz auf alternative Mittel zurückzugreifen. Und das

selbst dann, wenn diese auf den ersten Blick unorthodox und möglicherweise sogar etwas sperrig erscheinen.